# LSM Trees & SSTables

*memtable, compaction and bloom filters, versus B-trees*

Watch the 4-min video

**In one line:** an LSM tree turns every write into a log append plus a memory insert, flushes sorted immutable SSTables, and merges them in the background; writes are very fast, and bloom filters keep reads from checking every file.

## 1 A storage engine built for writes

A million IoT sensors each send a reading every 10 seconds: **100,000 writes per second**, all day, every day. A B-tree database would spend its time updating random pages on disk. Cassandra, RocksDB and ScyllaDB use a different storage engine built for exactly this: the **log-structured merge tree** (LSM tree).

## 2 The write path: one append, one memory write

1. Every write is first **appended** to a write-ahead log on disk, called the **commit log**, for durability. If the node crashes, the log is replayed.
2. Then it is inserted into the **memtable**, a sorted structure in memory.

That is it: one sequential append and one memory write. **No random disk I/O**, which is why writes are so fast.

When the memtable fills up (say at **64 MB**), it is flushed to disk as an **SSTable** (sorted string table): an **immutable** file of keys in sorted order, with a small index. Files are never modified in place. An **update** is simply a newer entry for the same key. A **delete** is written as a marker called a **tombstone**.

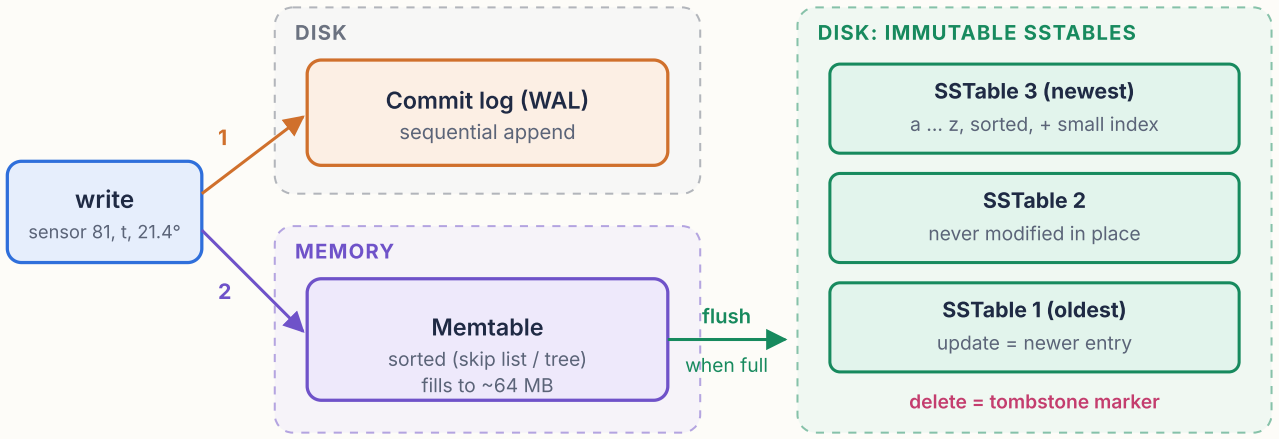



**Figure 1.** Writes touch only the end of the log and memory. Flushing writes a whole sorted file in one sequential pass. The commit log segment can be discarded once its memtable is flushed.

## 3 The read path: newest first, bloom filters skip the rest

Reads are the harder part. A key might be in the memtable or in any of many SSTables, so the engine checks the **newest first**, and the first hit wins.

To avoid opening every file, each SSTable has a **bloom filter**: a compact bit array that answers either *definitely not here* or *maybe here*. It never gives a false "no". With a **1% false-positive rate**, almost every

unnecessary file read is skipped. When it says “maybe”, a **sparse index** points to the right block inside the file.

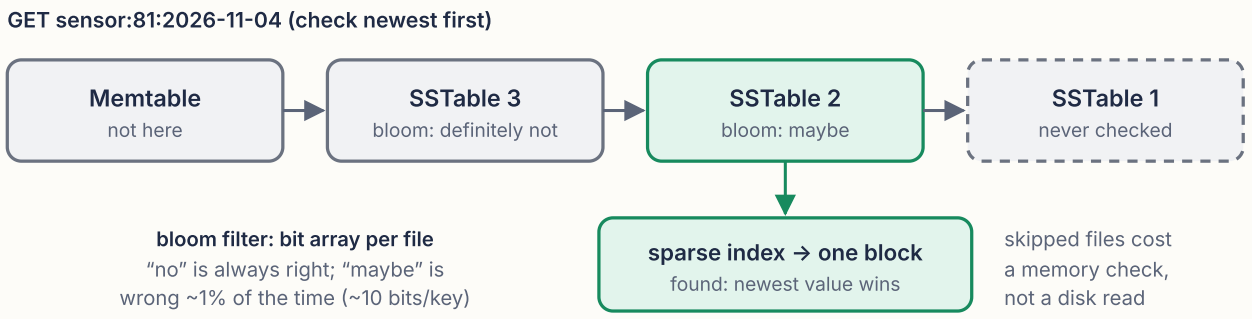



**Figure 2.** Only one file is actually read from disk. Bloom filters and sparse indexes are small enough to keep in memory. Bloom filters only help point lookups: a range scan must still merge every file that overlaps the range.

## 4 Compaction: merging the files

Over time SSTables pile up, full of old versions and tombstones. **Compaction** merges them in the background, like a merge sort: it keeps only the newest value per key and drops deleted data.

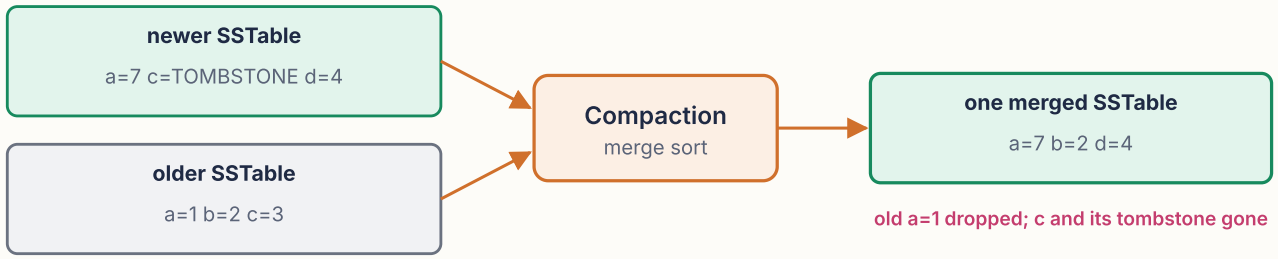



**Figure 3.** Because every input is already sorted, merging is one sequential pass. The tombstone can only be dropped once no older file could still hold the key.

| Strategy | How it works | Good at | Cost |
|---|---|---|---|
| **Size-tiered** | merge files of similar size | cheap for writes | more space; reads check more files |
| **Leveled** | within a level, key ranges never overlap | reads and space | more write amplification (data rewritten per level) |
| **Time-window** | group SSTables by time window | time series with TTL | bad if old data is updated |

## 5 B-tree versus LSM tree

| | B-tree | LSM tree |
|---|---|---|
| **Writes** | update pages in place: random I/O, page splits | sequential appends: very high throughput |
| **Reads** | fast and predictable: one place per key | may check several files; bloom filters help |
| **Space** | half-empty pages after splits | compresses well; old versions until compaction |
| **Background work** | little | compaction I/O can cause latency spikes |
| **Fits** | read-heavy OLTP: Postgres, MySQL | write-heavy: Cassandra, RocksDB, ScyllaDB |

## 6 Example: Cassandra ingesting IoT telemetry

WORKED EXAMPLE

- **Data model:** partition key `(sensor_id, day)`, clustering key `timestamp`. Each partition holds one sensor's day: 8,640 readings at one per 10 s.
- **Writes:** append-only, **100,000/s** spread across nodes by partition key. With 10 nodes and replication factor 3, each node takes about 100,000 × 3 / 10 = **30,000** writes/s, all sequential.
- **Compaction:** time-window, with a **90-day TTL**. Each window's SSTables hold only that window's data, so when every row in a file has expired, the whole file is dropped. No expensive per-row deletes.

COMMON MISTAKES

- Picking an LSM store for a read-heavy, update-heavy workload and fighting read amplification.
- Heavy deletes without thinking about tombstones: reads scan through thousands of them until compaction runs.
- Time-window compaction on data that is later updated or deleted out of order: old windows can never be dropped.
- Ignoring compaction when sizing disks: size-tiered compaction can temporarily need about twice the data size.

## 7 Key terms

**LSM tree**: log-structured merge tree: buffer, flush, merge

**Memtable**: sorted in-memory write buffer

**Tombstone**: marker that records a delete

**Compaction**: background merge of SSTables

**Commit log / WAL**: append-only log for durability

**SSTable**: immutable sorted file on disk

**Bloom filter**: “definitely not” or “maybe” per file

**Write / read / space amplification**: extra bytes written, files read, space used

## 8 Interview questions

**Q1. Compare B-tree and LSM tree storage engines.** BASIC
A B-tree keeps one sorted structure on disk and updates pages in place: fast, predictable reads, costlier random writes. An LSM tree buffers writes in a memtable backed by a WAL, flushes immutable SSTables and merges them by compaction, with bloom filters for reads. Very high write throughput, at the cost of read and space amplification and background compaction.

**Q2. Why are LSM writes so fast?** BASIC
Each write is one sequential append to the commit log plus an insert into the in-memory memtable. There is no random disk I/O on the write path; flushes and compaction write large sorted files sequentially.

**Q3. What does a bloom filter answer, and why does it help?** BASIC
Whether a key is “definitely not” in an SSTable or “maybe” in it. It never gives a false no. At a 1% false-positive rate (about 10 bits per key), a lookup across 10 files usually reads just one from disk.

**Q4. Design storage for 100,000 sensor writes per second with 90-day retention.** SCENARIO
Cassandra or ScyllaDB, partition by (sensor_id, day), cluster by timestamp, time-window compaction with a 90-day TTL so expired SSTables are dropped whole. That is about 8.6 billion rows a day; spread over 10 nodes with RF 3, each node takes about 30,000 sequential writes/s.

**Q5. Reads got slow on a Cassandra table after a large bulk delete. Why?** SCENARIO
Deletes are tombstones, not removals. Reads must skip every tombstone in the range until compaction purges them, which only happens after gc_grace_seconds (10 days by default). Avoid mass deletes; use TTLs or drop whole partitions or tables.

**Q6. p99 read latency on RocksDB spikes every few minutes. What is a likely cause?** SCENARIO
Compaction competing for disk I/O and CPU, or a write stall when L0 has too many files. Throttle compaction, add I/O capacity, or tune the strategy (leveled for read-heavy workloads).

**Q7. Why can't a tombstone be dropped the moment it is compacted?** TRICKY
An older SSTable, or a replica that missed the delete, may still hold the old value. If the tombstone vanished first, that old value would reappear. It must outlive every older copy of the key.

**Q8. Do bloom filters help range scans?** TRICKY
No. They answer membership of one exact key. A range scan must merge every SSTable whose key range overlaps; leveled compaction limits that to about one file per level.

Next: SD-E-007 · SQL Query Optimization backendroadmap.com